BIG DATA · INTERVIEW PREP

Hadoop MapReduce Interview Tutorial

A complete walkthrough of the MapReduce model — from the Word Count example to YARN architecture, fault tolerance, and the top interview questions.

Map & Reduce Phases | Shuffle & Sort | YARN Architecture | MapReduce vs Spark | Interview Q&A

1 What is MapReduce?

MapReduce is a distributed data processing programming model and execution framework used in Hadoop to process very large datasets across multiple machines.

It divides a large job into two main phases:

```
Input Data
    ↓
   MAP
    ↓
Intermediate Key-Value Pairs
    ↓
Shuffle and Sort
    ↓
   REDUCE
    ↓
Final Output
```

The basic idea is:

- **Map**: Transform input data into key-value pairs.
- **Reduce**: Aggregate values that have the same key.

2 Why Do We Need MapReduce?

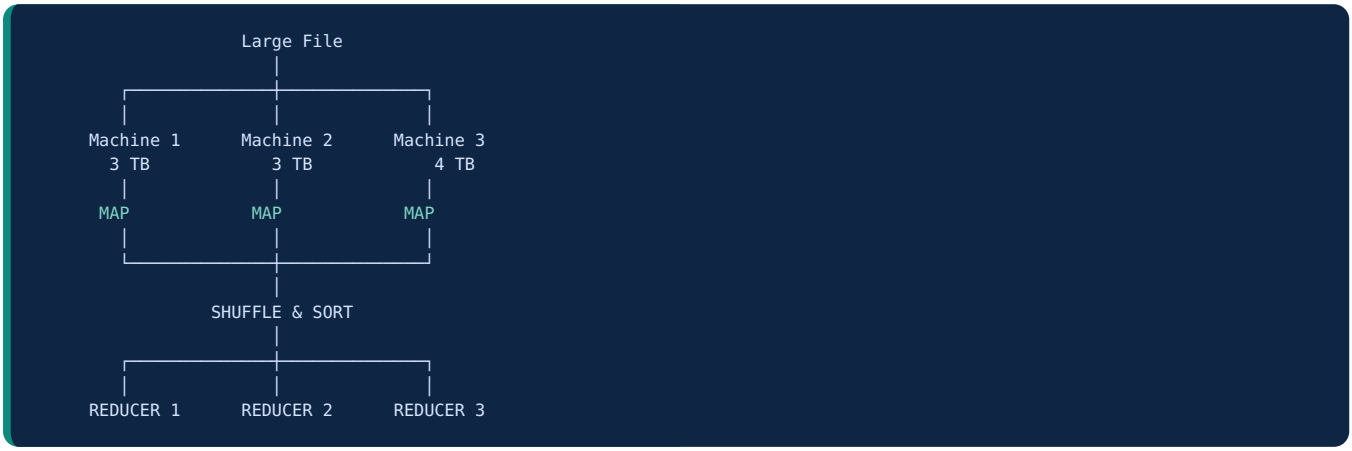
Suppose we have a file containing:

```
hello hadoop
hello spark
hadoop spark
```

We want to count every word. A traditional program might process the file using one machine:

```
Machine 1
    ↓
Read entire file
    ↓
Process entire file
    ↓
Return result
```

But imagine a file with **10 TB** of data — one machine would be too slow. Hadoop uses multiple machines:



This is called **parallel processing**.

3 The MapReduce Data Flow

The complete MapReduce flow is:

```
Input Data → InputFormat → Input Splits → RecordReader
    → Mapper → Combiner (Optional) → Partitioner
    → Shuffle and Sort → Reducer → OutputFormat → HDFS Output
```

> **🔑 Most important stages for an interview**
>
> Mapper → Combiner → Partitioner → Shuffle & Sort → Reducer

4 The Word Count Example

Suppose we have three files:

File	Content
File 1	hello hadoop
File 2	hello spark
File 3	hadoop spark

We want:

`hello 2` `hadoop 2` `spark 2`

5 Step 1: Input Data

Hadoop stores the data in HDFS. For example:

```
HDFS
│
├── Block 1
│   hello hadoop
│
├── Block 2
│   hello spark
│
└── Block 3
    hadoop spark
```

The data is divided into input splits. Usually:

> **Key point**
>
> 1 Input Split → 1 Mapper Task. An input split is a **logical division** of input data used by a mapper.

6 Step 2: Mapper

The Mapper receives data as <key, value>. For a text file:

- Key = line offset
- Value = line content

Example: `Key = 0, Value = "hello hadoop"`

The mapper splits the line into words:

```
Input:  hello hadoop
Output: <hello, 1>
        <hadoop, 1>
```

For all lines, the mapper produces:

Key	Value
hello	1
hadoop	1
hello	1
spark	1
hadoop	1
spark	1

7 Step 3: Combiner

The Combiner is an **optional local aggregation** step. Instead of sending <hello,1> <hello,1> over the network, the Combiner can calculate <hello,2>, reducing network traffic.

```
Mapper Output:              Combiner:
<hello, 1>                  <hello, 2>
<hello, 1>                  <hadoop, 2>
<hadoop, 1>
<hadoop, 1>
```

> ⚠ **Important interview question — Is Combiner guaranteed to run?**
>
> **No.** Hadoop may execute it 0 times, 1 time, or multiple times. Therefore, the Combiner should only be used when the operation is safe for repeated execution.
>
> ✅ Good example: **SUM** | ❌ Dangerous example: **AVERAGE** (averaging partial averages produces wrong results).

8 Step 4: Partitioner

The Partitioner determines **which Reducer should receive each key**. Suppose we have 3 reducers (Reducer 0, 1, 2). The default partitioner usually uses:

```
hash(key) % number_of_reducers
```

Key	Reducer
hello	Reducer 1
hadoop	Reducer 0
spark	Reducer 2

> 🔑 **Key point**
>
> The same key must always go to the same Reducer, because the Reducer needs all values for that key together: `hello → [1, 1]`

9 Step 5: Shuffle and Sort

This is **one of the most important phases** in MapReduce. The Mapper output gets grouped by key:

```
Before                      After Shuffle & Sort
<hello, 1>                  <hadoop, [1, 1]>
<hadoop, 1>                 <hello,  [1, 1]>
<hello, 1>                  <spark,  [1, 1]>
<spark, 1>
<hadoop, 1>
<spark, 1>
```

The Reducer does **not** receive individual records like `<hello,1> <hello,1>` — instead it receives `hello → [1, 1]`.

10 Step 6: Reducer

The Reducer aggregates the values using `sum(values)`:

```
<hadoop, [1, 1]>  →  <hadoop, 2>
<hello,  [1, 1]>  →  <hello, 2>
<spark,  [1, 1]>  →  <spark, 2>
```

11 Complete Word Count Flow

```
INPUT
─────────────────────────
hello hadoop
hello spark
hadoop spark
        ↓
MAPPER
─────────────────────────
<hello,1> <hadoop,1> <hello,1> <spark,1> <hadoop,1> <spark,1>
        ↓
COMBINER
─────────────────────────
<hello,2> <hadoop,2> <spark,2>
        ↓
SHUFFLE & SORT
─────────────────────────
<hadoop,[2]> <hello,[2]> <spark,[2]>
        ↓
REDUCER
─────────────────────────
<hadoop,2> <hello,2> <spark,2>
        ↓
OUTPUT TO HDFS
─────────────────────────
hadoop  2
hello   2
spark   2
```

12 MapReduce Components

1. InputFormat

Determines how the input data is split and how records are read. Default: **TextInputFormat**, producing `<line offset, line text>`, e.g. `<0, "hello hadoop">`.

2. InputSplit

A logical portion of the input data.

```
Large File: 1 GB
Input Split 1 → 128 MB
Input Split 2 → 128 MB
Input Split 3 → 128 MB ...
```

> 📌 **Important**
>
> Number of Mappers is usually related to the number of InputSplits, not directly to the number of HDFS blocks.

3. RecordReader

Converts the input split into key-value pairs: `Input: "hello hadoop" → Key=0, Value="hello hadoop"`

4. Mapper

Transforms input records: `<0, "hello hadoop"> → <hello,1> <hadoop,1>`

5. Combiner

Performs local aggregation (optional): `<hello,1> <hello,1> → <hello,2>`

6. Partitioner

Decides which Reducer receives each key: `hash(key) % number_of_reducers`

7. Shuffle and Sort

Responsible for moving Mapper output, grouping identical keys, and sorting keys.

8. Reducer

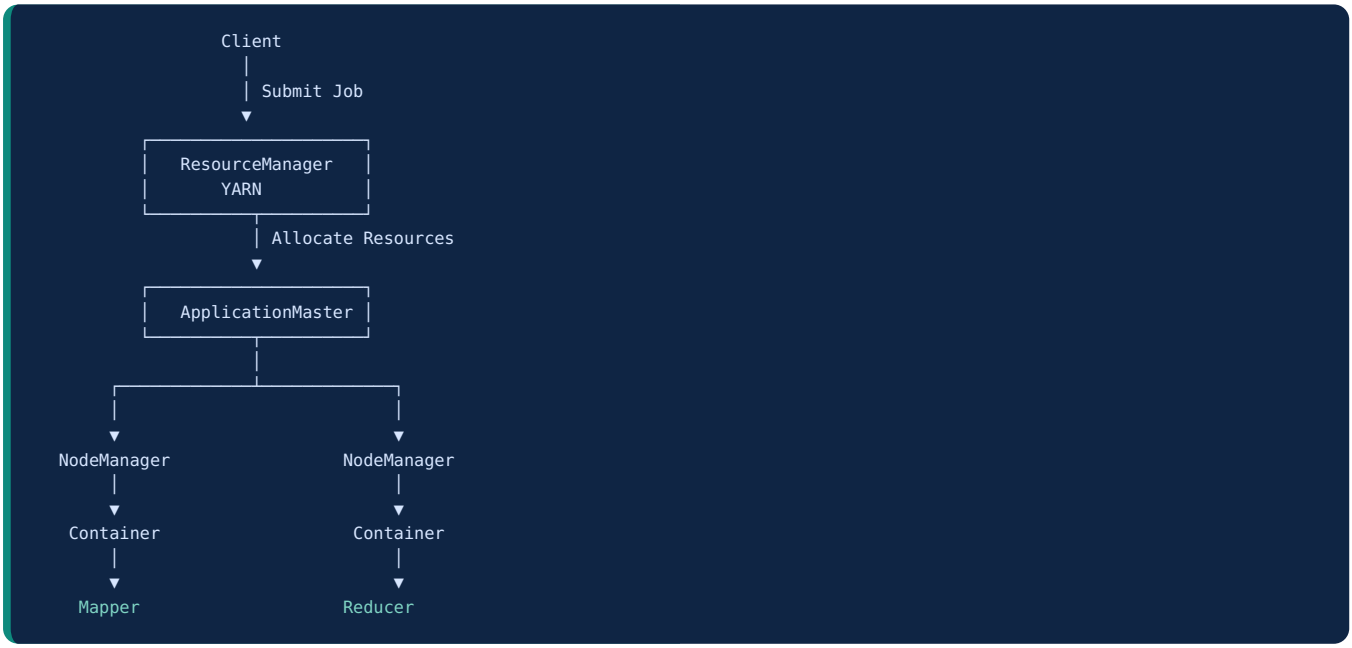
Processes grouped data: `hello → [1,1] → hello → 2`

9. OutputFormat

Writes the final output to HDFS. Default: **TextOutputFormat**, format key<TAB>value.

13 MapReduce Architecture

In modern Hadoop, MapReduce runs on top of **HDFS + YARN + MapReduce**:



14 How a MapReduce Job Runs

1. **Client submits the job** — the client sends a MapReduce Job to the ResourceManager.
2. **ResourceManager starts ApplicationMaster** — manages the specific job and requests resources from YARN.
3. **Input data is divided** into Input Splits, e.g. Split 1 → Mapper 1, Split 2 → Mapper 2, Split 3 → Mapper 3.
4. **YARN allocates containers** for Mapper Tasks and Reducer Tasks.
5. **Mappers process data**: Input Split → Mapper → Intermediate Key-Value Pairs.
6. **Shuffle and Sort** — Mapper output is transferred to the appropriate Reducer.
7. **Reducers produce final output** written to HDFS, e.g. `/user/youssef/output/part-r-00000`

15 Map Task vs Reduce Task

Map Task	Reduce Task
Processes input data	Processes intermediate data
Runs close to input data	Receives data from Mappers
Produces key-value pairs	Aggregates values
Usually more numerous	Usually fewer
First major processing phase	Final major processing phase

```
Map:     "hello world" → hello→1, world→1
Reduce:  hello→[1,1,1] → hello→3
```

16 What Happens If a Node Fails?

Hadoop MapReduce provides **fault tolerance**. Suppose Mapper 1 fails. The ApplicationMaster detects the failure, then Hadoop:

1. Detects the failed task
2. Starts the task again
3. Runs it on another available node

Because the input data is stored in HDFS with replication:

```
Block Replica 1 → DataNode 1
Block Replica 2 → DataNode 2
Block Replica 3 → DataNode 3
```

> 📌 **Very important interview point**
>
> Hadoop achieves fault tolerance through **HDFS data replication** and **MapReduce task re-execution**.

17 Data Locality

Hadoop tries to move **Computation → Data** instead of **Data → Computation**.

```
Bad Approach:              Hadoop Approach:
DataNode 1                 DataNode 1
   | Transfer 1 TB             |
   ▼                           └── Run Mapper locally
Processing Node            (very cheap)
(very expensive)
```

This is called **data locality**. Levels, best to worst:

1. **Node Local** (best)
2. Rack Local
3. Off Rack

18 Important MapReduce Interview Questions

Q1. What is MapReduce?

MapReduce is a distributed processing framework and programming model in Hadoop used to process large datasets in parallel across multiple nodes. It consists mainly of a Map phase, which transforms input data into intermediate key-value pairs, followed by Shuffle and Sort, which groups data by key, and a Reduce phase, which aggregates the grouped values.

Q2. What is the difference between Mapper and Reducer?

The Mapper processes input data and generates intermediate key-value pairs. The Reducer receives all values associated with the same key after the shuffle and sort phase and performs aggregation or final processing.

Q3. What is Shuffle and Sort?

The phase that transfers Mapper output to the appropriate Reducers, groups records with the same key, and sorts the keys before the Reducer processes them.

Q4. What is a Combiner?

An optional local aggregation function that runs after the Mapper and before the Shuffle phase. It reduces the amount of data transferred across the network.

Q5. Is Combiner guaranteed to execute?

No. Hadoop may execute the Combiner zero, one, or multiple times. Therefore, the Combiner logic must be safe for repeated execution.

Q6. How many Mappers are created?

Mainly determined by the number of InputSplits. Usually 1 InputSplit ≈ 1 Mapper.

Q7. How many Reducers are created?

Configured by `mapreduce.job.reduces`, e.g. `-D mapreduce.job.reduces=3`. If Number of Reducers = 0, the job is a **Map-only Job** — there is no Shuffle, Sort, or Reduce phase.

Q8. What happens if there are no Reducers?

The flow becomes Input → Mapper → Output directly (e.g. CSV → Mapper → Cleaned CSV). This is called a **Map-only job**.

Q9. What is the default number of Reducers?

Default Reducers = 1, but in real projects the number is configured according to the workload.

Q10. Why is MapReduce slow compared to Spark?

MapReduce usually writes intermediate data to disk between stages (Map → Disk → Shuffle → Disk → Reduce), while Spark can keep intermediate data in memory (Map → Memory → Transformation → Reduce). MapReduce is disk-based processing; Spark is memory-based processing — though Spark can also use disk when necessary.

19 MapReduce vs Spark

Feature	MapReduce	Spark
Processing	Disk-based	In-memory primarily
Speed	Slower	Faster
Programming	More complex	Easier
Iterative algorithms	Poor	Excellent
Machine Learning	Less suitable	Better
Streaming	Limited	Better
Fault tolerance	Replication + re-execution	Lineage + re-computation

🔑 Interview answer

MapReduce is disk-oriented and processes data in distinct Map and Reduce stages. Spark uses in-memory computation and supports more flexible processing models such as batch processing, streaming, SQL, machine learning, and graph processing.

20 The Most Important Diagram to Memorize

★ Your Interview Study Order

Level 1 — Understand the concept

What is MapReduce? · Why do we need it? · Map vs Reduce

Level 2 — Understand the flow

Input → InputSplit → Mapper → Combiner → Partitioner → Shuffle & Sort → Reducer → Output

Level 3 — Understand Hadoop integration

HDFS → YARN → MapReduce

Level 4 — Practice: build these projects

`Word Count` `Average Temperature` `Maximum Temperature` `Sales Aggregation` `Customer Count by Country` `Log Analysis` `Top 10 Products by Sales`

Next step: Since you are currently practicing Hadoop and PySpark, the best next step is to build a complete MapReduce project using a real CSV dataset, then compare the same project with PySpark.